

Indian Institute of Space Science and Technology
Thiruvananthapuram



Course on Python Programming, Data Processing & Visualization

(28 September – 09 October 2026)

Python Fundamentals

Introduction · Variables · Data Types · Constants · Type Conversion · Operators

Diya P M

Department of Avionics

Indian Institute of Space Science and Technology, Thiruvananthapuram

29 September 2026

Day 2 – Session 1

Introduction to Python

- Python is a **high-level, general-purpose programming language**.
- Created by **Guido van Rossum** and first released in 1991.
- Designed with an emphasis on **readability, simplicity, and productivity**.
- Python programs are commonly used for:
  - Web development
  - Data science and data analysis
  - Machine learning and artificial intelligence
  - Automation and scripting
  - Scientific computing
  - Signal and image processing

Why Learn Python?

- **Simple syntax:** Python code is close to natural language.
- **Readable:** Programs are generally easy to understand and maintain.
- **Free and open source:** Python can be used without purchasing a license.
- **Portable:** Python is available on Windows, Linux and macOS.
- **Large ecosystem:** Libraries such as NumPy, Pandas, Matplotlib and PyTorch support many applications.
- **Large community:** Many tutorials, books, forums and documentation resources are available.

First Python Program

The simplest Python program is:

```
print("Hello, World!")
```

Output

```
Hello, World!
```

- `print()` is a built-in Python function.
- The text inside quotation marks is a string.
- A semicolon is not required at the end of every statement.

```
print("Welcome to Python")  #      Welcome to Python
print(10)                   #       10
print(10 + 5)               #       15
```

Comments in Python

Comments are notes written in a program for explanation.

- A single-line comment begins with #.
- The interpreter does not execute the comment as a Python statement.

```
# Store the student's marks
marks = 85

# Display the marks
print(marks)
```

Why use comments?

Comments explain the purpose of code, document important steps, and make programs easier for others to read.

What is a Variable?

A **variable** is a name that refers to a value.

```
age = 25
name = "Bob"
height = 170.5
```

Here:

- `age`, `name`, and `height` are variable names.
- `25`, `"Bob"`, and `170.5` are values.
- `=` is the assignment operator.

Simple idea

A variable gives a meaningful name to data so that the data can be used later.

Why Are Variables Used?

Variables allow us to **store, reuse, modify, and process** data.

```
length = 10
width = 5

area = length * width
print(area)
```

- `length` stores the length.
- `width` stores the width.
- `area` stores the calculated result.

If the width changes:

```
width = 8
area = length * width
```

The same calculation can be reused with a different value.

Assignment and Reassignment

The = operator assigns a value to a variable.

```
x = 10
```

The variable can later be assigned another value:

```
x = 10
x = 20
```

After the second statement:

$$x = 20$$

This is called **reassignment**.

Important

`x = 20` does not mean "x is equal to 20" in the mathematical sense. It means "assign the value 20 to x."

Variable Naming Rules – Overview

Python variable names must follow syntactic rules.

1. A name may contain **letters, digits and underscores**.
2. A name **cannot start with a digit**.
3. Variable names are **case-sensitive**.
4. A variable name **cannot be a Python keyword**.
5. Spaces and most special characters are not allowed.

Good practice

Use meaningful names such as `student_name`, `total_marks`, and `average_score`.

Rule 1: Letters, Digits and Underscore

A variable name may contain:

- Uppercase letters: `A-Z`
- Lowercase letters: `a-z`
- Digits: `0-9`
- Underscore: `_`

Valid examples

```
age
student1
student_name
marks2026
_value
```

Not allowed

Characters such as `-`, `@`, `$`, `#` and spaces cannot normally be used in a variable name.

Rule 1: Examples in Code

Valid:

```
student_name = "Anu"
marks1 = 90
total_marks2026 = 450
_temp = 10
```

Invalid:

```
student-name = "Anu"    # - is not allowed
student name = "Anu"    # space is not allowed
total$ = 100             # $ is not allowed
```

Remember

Underscore is allowed and is commonly used to separate words.

Rule 2: A Variable Name Cannot Start with a Digit

A variable name may contain digits, but it **cannot begin with a digit**.

Valid

```
marks1
student2
data2026
value_1
```

Invalid

```
1marks
2student
2026data
```

- The problem is specifically the **first character**.
- Digits are perfectly valid after the first character.

Rule 3: Variable Names Are Case-Sensitive

Python treats uppercase and lowercase letters as different characters.
Therefore:

```
age
Age
AGE
aGe
```

Example

```
age = 20
Age = 30
print(age)   # 20
print(Age)   # 30
```

Common mistake

Changing the capitalization accidentally can produce a `NameError` if the intended variable does not exist.

Rule 4: Python Keywords Cannot Be Variable Names

Python reserves certain words for its own syntax. These are called **keywords**.
Examples include:

```
if  else  for  while  class  def  return  import
```

They cannot be used as ordinary variable names.

Invalid

```
class = 10
for = 5
if = 20
```

Rule 4: Keyword Examples

Incorrect:

```
class = 10
for = 20
if = 30
```

Python reports a syntax error because these words have predefined meanings.

Use alternatives:

```
class_name = "ECE"
for_count = 20
if_value = 30
```

How to check keywords

```
import keyword
print(keyword.kwlist)
```

Rule 5: Spaces and Special Characters

Spaces are not allowed inside a variable name.

Invalid

```
student name
total marks
phone number
```

Use underscores instead:

Valid

```
student_name
total_marks
phone_number
```

Similarly, avoid symbols such as:

`@, $, #, -, !`

Good Naming Practices

Rules determine whether a name is **valid**; good practices make a valid name **readable**.

- Use descriptive names.
- Use **snake_case** for ordinary variables.
- Avoid unnecessary one-letter names except for simple cases such as loop counters.
- Avoid confusing names such as `l`, `I`, and `O`.
- Do not use names that hide the purpose of the data.

Prefer

```
total_marks = 450
over
x = 450
```

Valid vs Invalid Variable Names

Name	Status	Reason
`student_name`	Valid	Letters and underscore
`marks1`	Valid	Digit is not first
`_total`	Valid	Underscore is allowed
`1marks`	Invalid	Starts with a digit
`student-name`	Invalid	Hyphen is not allowed
`student name`	Invalid	Contains a space
`class`	Invalid	Python keyword
`StudentName`	Valid	Case is allowed
`total$`	Invalid	Special character

Multiple Assignment

Python allows several variables to be assigned in one statement.

```
x, y, z = 10, 20, 30
```

This is equivalent to:

```
x = 10
y = 20
z = 30
```

The number of variables and values must match.

```
a = b = c = 0
```

Here all three variables receive the value 0.

Swapping Variables

Python can swap two variables without a temporary variable.

```
a = 10
b = 20

a, b = b, a

print(a)  # 20
print(b)  # 10
```

Why useful?

This is a common example of Python's convenient multiple-assignment syntax.

Data Types in Python

A **data type** describes the kind of value stored or represented.
Common built-in types include:

- `int` – integers
- `float` – floating-point numbers
- `complex` – complex numbers
- `str` – text
- `bool` – Boolean values
- `list` – ordered, mutable collection
- `tuple` – ordered, immutable collection
- `set` – collection of unique elements
- `dict` – key-value pairs
- `NoneType` – the type of `None`

Integer Data Type – int

The `int` type represents whole numbers.

```
age = 25
temperature = -10
count = 100
zero = 0
```

Integers can be:

- Positive
- Negative
- Zero

Examples

$$25, \quad -10, \quad 0, \quad 100, \quad 2026$$

Floating-Point Data Type – float

The `float` type represents real-valued numbers, commonly written with a decimal point.

```
height = 175.5
temperature = 36.7
pi = 3.14159
```

Examples:

$$3.14, \quad -2.5, \quad 0.75, \quad 10.0$$

Important

`10` is an integer, while `10.0` is a float.

Complex Data Type – complex

A complex number has the form:

$$a + bj$$

where:

- a is the real part.
- b is the imaginary part.
- j is Python's imaginary unit.

Example:

```
z = 3 + 4j
```

The real and imaginary parts can be obtained using:

```
print(z.real)
print(z.imag)
```

Output:

```
3.0
4.0
```

String Data Type – str

A string is a sequence of characters.
Strings can use single or double quotation marks:

```
name = "Python"
course = 'DSP'
```

A string can contain letters, digits, spaces and special characters:

```
message = "Welcome to Python!"
code = "ECE2026"
```

Important

"25" is a string, while 25 is an integer.

Boolean Data Type – bool

The Boolean type has two values:

- `True`
- `False`

Example:

```
is_student = True
is_finished = False
```

Booleans commonly appear as the results of comparisons:

```
age = 25
print(age >= 18)
```

Output:

```
True
```

List in Python

A **list** is a collection used to store multiple values in one variable.

- Lists are **ordered**.
- Lists are **mutable**, meaning they can be changed.
- Lists allow **duplicate values**.
- Lists can contain different data types.
- Lists are written using square brackets `[ ]`.

Example:

```
marks = [80, 90, 75, 88]
```

The four values are stored together in one list.

When to use a list

Use a list when you have a collection of values that may need to be modified.

Creating Lists

A list is created using square brackets.

```
numbers = [10, 20, 30, 40]

names = ["Anu", "Ravi", "Meera"]

marks = [80, 75, 90, 88]
```

A list can contain different types of values:

```
student = ["Bob", 25, 175.5, True]
```

Here the list contains:

- String: `"Bob"`
- Integer: 25
- Float: `175.5`
- Boolean: `True`

List Indexing

Python uses **zero-based indexing**.
Consider:

```
marks = [80, 90, 75, 88]
```

Index	Value
0	80
1	90
2	75
3	88

Therefore:

```
print(marks[0])    # 80
print(marks[2])    # 75
```

Remember

The first element always has index 0.

Negative Indexing in Lists

Python also supports negative indexing.
Negative indexing starts from the end.

```
marks = [80, 90, 75, 88]
```

Index	Value
-1	88
-2	75
-3	90
-4	80

Example:

```
print(marks[-1])   # 88
print(marks[-2])   # 75
```

Lists are Mutable

Lists are **mutable**.
This means that we can change an existing element after creating the list.

```
marks = [80, 90, 75]

marks[1] = 95

print(marks)
```

Output:

```
[80, 95, 75]
```

The value at index 1 changed from 90 to 95.

Key idea

Mutable means that the contents of an object can be modified.

Adding and Removing List Elements

An element can be added using `append()`.

```
numbers = [10, 20, 30]

numbers.append(40)

print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

An element can be removed using `remove()`.

```
numbers.remove(20)
print(numbers)
```

Output: [10, 30, 40]

List Slicing

List slicing is used to obtain a part of a list.

General syntax:

```
list[start:stop]
```

Example

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])
```

Output:

```
[20, 30, 40]
```

Important Rule

The **starting index is included**, but the **ending index is excluded**.

Tuple in Python

A **tuple** is a collection used to store multiple values.

- Tuples are **ordered**.
- Tuples are **immutable**.
- Tuples allow duplicate values.
- Tuples can contain different data types.
- Tuples are commonly written using parentheses `( )`.

Example:

```
coordinates = (10, 20)
```

Main idea

Use a tuple when the values represent a collection that should not normally be changed.

Creating and Accessing Tuples

Example:

```
marks = (80, 90, 75, 88)
```

Tuples support indexing:

```
print(marks[0])    # 80
print(marks[2])    # 75
```

Negative indexing is also possible:

```
print(marks[-1])   # 88
```

A tuple can contain different data types:

```
student = ("Bob", 25, 170.5, True)
```

Tuples are Immutable

Tuples cannot be modified after they are created.
For example:

```
numbers = (10, 20, 30)

numbers[1] = 50
```

The second statement produces an error because tuples are immutable.
Compare this with a list:

```
numbers = [10, 20, 30]

numbers[1] = 50
```

The list can be modified.

Remember

List → mutable
Tuple → immutable

Set in Python

A **set** is a collection that stores **unique values**.
Properties of sets:

- Duplicate values are automatically removed.
- Sets do not support normal numerical indexing.
- Sets are mutable.
- Sets are useful for membership testing.
- Sets are written using curly braces `{ }`.

Example:

```
numbers = {10, 20, 30}
```

Main idea

Use a set when uniqueness is important.

Sets Automatically Remove Duplicates

Consider:

```
numbers = {10, 20, 20, 30, 30, 30}

print(numbers)
```

The duplicate values are removed.
The resulting set contains only the unique values:

```
{10, 20, 30}
```

A practical example:

```
marks = [80, 90, 80, 75, 90]
unique_marks = set(marks)
print(unique_marks)
```

The set contains each distinct value only once.

Adding and Removing Set Elements

An element can be added using `add()`.

```
numbers = {10, 20, 30}

numbers.add(40)

print(numbers)
```

An element can be removed using `remove()`.

```
numbers.remove(20)

print(numbers)
```

Important

Sets are not indexed like lists or tuples, so expressions such as `numbers[0]` should not be used.

Set Operations

Sets support mathematical operations.
Consider:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

- Union: `A | B`
- Intersection: `A & B`
- Difference: `A - B`
- Symmetric difference: `A ^ B`

These operations are useful for comparing groups of values.

Set Operations: Example

Let

$$A = \{1, 2, 3, 4\}, \qquad B = \{3, 4, 5, 6\}$$

Operation	Result	Meaning
`A | B`	$\{1, 2, 3, 4, 5, 6\}$	All elements
`A & B`	$\{3, 4\}$	Common elements
`A - B`	$\{1, 2\}$	In A, not in B

Remember

`|` = Union `&` = Intersection `-` = Difference

Dictionary in Python

A **dictionary** stores information as:

key : value

Example:

```
student = {"name":  "Bob", "age":  25, "marks":  85}
```

Here:

- `"name"` is a key.
- `"Bob"` is its value.
- `"age"` is a key.
- 25 is its value.
- `"marks"` is a key.
- 85 is its value.

Main idea

A dictionary is useful when each piece of data has a meaningful label.

Accessing Dictionary Values

Dictionary values are accessed using their keys.

```
student = {
 "name": "Bob",
 "age": 25,
 "marks": 85
}

print(student["name"])
print(student["marks"])
```

Output:

```
Bob
85
```

Unlike a list, we normally access dictionary values using meaningful keys rather than numerical indices.

Adding and Updating Dictionary Values

A dictionary value can be **added** or **updated** using the same syntax:

```
dictionary[key] = value
```

Example

```
student = {"name": "Bob", "marks": 85}
student["marks"] = 92   # Existing key → Update
student["age"] = 25   # New key → Add
print(student)
```

Result:

```
{"name": "Bob", "marks": 92, "age": 25}
```

Remember

Key exists → Value is updated **Key does not exist** → New pair is added

Removing Dictionary Elements

The `pop()` method can be used to remove a key-value pair.

```
student = {
 "name": "Bob",
 "age": 25,
 "marks": 85
}

student.pop("age")

print(student)
```

The resulting dictionary is:

```
{"name": "Bob", "marks": 85}
```

The key and its corresponding value are removed together.

None in Python

`None` represents the **absence of a value**.
Example:

```
result = None
```

This means that the variable exists, but no meaningful value has been assigned to it yet.

- `None` is a special Python value.
- Its data type is `NoneType`.
- It is commonly used when a value is unavailable.
- It is also commonly used as a default or placeholder value.

Important

`None` does not mean zero.

None vs Zero vs Empty String

These three values are different:

```
a = None
b = 0
c = ""
```

Their types are:

```
print(type(a))   # NoneType
print(type(b))   # int
print(type(c))   # str
```

- `None` → absence of a value
- `0` → integer value zero
- `""` → empty string

Remember

None, zero, and an empty string are not the same thing.

Checking the Data Type with type()

Python provides the `type()` function.

```
x = 25
print(type(x))
```

Output:

```
<class 'int'>
```

More examples:

```
print(type(3.14))       # float
print(type("Python"))   # str
print(type(True))       # bool
print(type([1, 2, 3]))  # list
```

Constants in Python

A **constant** is a value that is intended to remain unchanged during a program.
Python does not have a special `const` keyword for ordinary constants.
Instead, uppercase names are used by convention:

```
PI = 3.14159
MAX_SIZE = 100
GRAVITY = 9.81
```

Constants: Example

```
PI = 3.14159
radius = 5

area = PI * radius ** 2
print(area)
```

Here:

- `PI` represents a value intended to stay fixed.
- `radius` is an ordinary variable.
- `area` is calculated from both.

The naming convention helps other programmers recognize values that should not normally be changed.

Variable vs Constant

Feature	Variable	Constant convention
Purpose	May change	Intended to remain fixed
Naming	Usually lowercase/snake_case	Usually UPPERCASE
Example	`age = 25`	`PI = 3.14`

Type Conversion

Type conversion means changing a value from one data type to another.
Common conversion functions:

- `int()`
- `float()`
- `str()`
- `bool()`

Example:

```
x = "25"
y = int(x)
```

Now:

- `x` is a string.
- `y` is an integer.

int() Conversion

`int()` can convert suitable values to integers.

```
a = int("25")       # 25
b = int(3.9)        # 3
c = int(-2.7)       # -2
```

Important

Converting a float to an integer removes the fractional part; it does not round to the nearest integer.

For example:

$$\texttt{int(3.9)} = 3$$

$$\texttt{int(-3.9)} = -3$$

float() and str() Conversion

`float()`:

```
x = float("3.14")
y = float(10)
```

Results:

3.14, 10.0

`str()`:

```
age = 25
message = "Age: " + str(age)
print(message)
```

Output:

```
Age: 25
```

bool() Conversion

`bool()` converts a value to a Boolean value.
Common examples:

```
bool(1)       # True
bool(0)       # False
bool(25)      # True
bool("")      # False
bool("Hello") # True
bool([])      # False
```

Idea

Several values are considered “false-like”, including `0`, `False`, `None`, and empty collections.

Input and Type Conversion

The `input()` function receives input from the user.

```
age = input("Enter your age: ")
```

The value returned by `input()` is a **string**.
Therefore, numerical input is usually converted:

```
age = int(input("Enter your age: "))
height = float(input("Enter height: "))
```

This allows the values to be used in numerical calculations.

Input Example: Calculate Area

```
length = float(input("Enter length: "))
width = float(input("Enter width: "))

area = length * width

print("Area =", area)
```

Example interaction:

```
Enter length: 10
Enter width: 5
Area = 50.0
```

Concept

Input → conversion → calculation → output

Operators in Python

An **operator** is a symbol or keyword used to perform an operation.
Example:

```
a = 10
b = 5
result = a + b
```

Here:

- `a` and `b` are operands.
- `+` is the operator.
- `a + b` is an expression.
- `result` stores the result.

Types of Operators

Python provides several categories of operators:

- Arithmetic
- Assignment
- Comparison
- Logical
- Membership
- Identity
- Bitwise

These operators are used for calculations, updating variables, comparing values, combining conditions, checking membership, checking object identity, and manipulating bits.

Arithmetic Operators

Operator	Operation	Example	Result
+	Addition	`10 + 3`	13
-	Subtraction	`10 - 3`	7
*	Multiplication	`10 * 3`	30
/	Division	`10 / 3`	3.333...
//	Floor division	`10 // 3`	3
%	Modulus	`10 % 3`	1
**	Exponentiation	`10 ** 3`	1000

Arithmetic Operators: Detailed Example

```
a = 10
b = 3

print(a + b)   # 13
print(a - b)   # 7
print(a * b)   # 30
print(a / b)   # 3.333...
print(a // b)  # 3
print(a % b)   # 1
print(a ** b)  # 1000
```

- / gives ordinary division.
- // gives floor division.
- % gives the remainder.
- ** calculates a power.

Modulus Operator: Practical Examples

The modulus operator % gives the remainder.

```
17 % 5
```

Since:

$$17 = 5 \times 3 + 2$$

the result is:

$$17\%5 = 2$$

Checking even and odd numbers:

```
number = 8

print(number % 2 == 0)
```

Output:

```
True
```

Floor Division: Practical Example

Floor division uses //.

```
10 // 3
```

Ordinary division:

$$10/3 = 3.333\ldots$$

Floor division:

$$10//3 = 3$$

Another example:

```
25 // 4
```

gives:

6

because the floor of 6.25 is 6.

Assignment Operators

Assignment operators assign or update values.

Operator	Meaning	Example
=	Assign	`x = 10`
+=	Add and assign	`x += 5`
-=	Subtract and assign	`x -= 5`
*=	Multiply and assign	`x *= 5`
/=	Divide and assign	`x /= 5`
%=	Modulus and assign	`x %= 5`
**=	Power and assign	`x **= 2`

Assignment Operators: Example

```
x = 10

x += 5      # x = x + 5  -> 15
x -= 3      # x = x - 3  -> 12
x *= 2      # x = x * 2  -> 24
x //= 5     # x = x // 5 -> 4
```

The shortened form is called a **compound assignment operator**.

Example

`x += 5` is equivalent to `x = x + 5`.

Comparison Operators

Comparison operators compare two values.
The result is a Boolean value: `True` or `False`.

Operator	Meaning	Example
==	Equal to	5 == 5
!=	Not equal to	5 != 3
>	Greater than	5 > 3
<	Less than	3 < 5
>=	Greater/equal	5 >= 5
<=	Less/equal	3 <= 5

Comparison Operators: Examples

```
x = 10

print(x == 10)   # True
print(x != 10)   # False
print(x > 5)     # True
print(x < 5)     # False
print(x >= 10)   # True
print(x <= 9)    # False
```

Comparison expressions are especially useful in:

- `if` statements
- Loops
- Filtering data
- Decision making

Important: = vs ==

These operators have completely different purposes.
= is assignment:

```
x = 10
```

It stores 10 in `x`.
== is comparison:

```
x == 10
```

It checks whether the value of `x` is equal to 10.

Remember

= → assign
== → compare

Logical Operators

Logical operators combine or modify conditions.

- `and`: both conditions must be true.
- `or`: at least one condition must be true.
- `not`: reverses a Boolean result.

Example:

```
age = 25
print(age > 18 and age < 60)
```

Both conditions are true, so the result is `True`.

Logical Operators: Truth Table

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

For `not`:

- `not True` → `False`
- `not False` → `True`

Logical Operators: Practical Example

Suppose a student passes only when:

- marks are at least 40, and
- attendance is at least 75.

```
marks = 65
attendance = 80

passed = marks >= 40 and attendance >= 75
print(passed)
```

Output:

```
True
```

This shows how logical operators combine multiple conditions.

Membership Operators

Membership operators check whether a value exists inside a collection or sequence.
Python provides:

- `in`
- `not in`

Example:

```
numbers = [10, 20, 30]

print(20 in numbers)       # True
print(40 in numbers)       # False
print(40 not in numbers)   # True
```

Membership Operators with Strings

Membership operators can also be used with strings.

```
message = "Python programming"

print("Python" in message)
print("Java" in message)
print("Java" not in message)
```

Output:

```
True
False
True
```

Use

Membership testing is useful when checking whether an item or substring exists in a collection.

Identity Operators

Identity operators check whether two variables refer to the **same object**.
Python provides:

- `is`
- `is not`

Example:

```
a = [1, 2]
b = a

print(a is b)
```

Output:

```
True
```

Both names refer to the same list object.

Equality vs Identity

== checks whether values are equal.
`is` checks whether two references point to the same object.

```
a = [1, 2]
b = [1, 2]

print(a == b)
print(a is b)
```

The first comparison asks:

Do the two lists contain equal values?

The second asks:

Are they the exact same object?

Good practice

Use == for value equality and `is` mainly for identity checks such as `x is None`.

Bitwise Operators

Bitwise operators operate on individual bits of integer values.

Operator	Operation
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
«	Left shift
»	Right shift

These are particularly relevant when working with binary data, low-level programming, embedded systems, and some signal/data processing tasks.

Bitwise AND and OR Example

Consider:

$$5 = 101_2, \qquad 3 = 011_2$$

Bitwise AND:

$$101 \ \& \ 011 = 001$$

Therefore:

$$5\&3 = 1$$

Bitwise OR:

$$101 \mid 011 = 111$$

Therefore:

$$5|3 = 7$$

Bitwise XOR and NOT

For XOR, the output bit is 1 when the input bits are different.

$$101 \oplus 011 = 110$$

Therefore:

$$5 \hat{} \ 3 = 6$$

For bitwise NOT:

```
~5
```

Python uses signed integer representation, so the result may look surprising to beginners. Bitwise NOT is effectively related to:

$$\sim x = -x - 1$$

For example:

$$\sim 5 = -6$$

Bitwise Shift Operators

Shift operators move bits left or right.
For:

$$8 = 1000_2$$

Left shift:

$$1000 \ll 1 = 10000$$

so:

$$8 \ll 1 = 16$$

Right shift:

$$1000 \gg 1 = 0100$$

so:

$$8 \gg 1 = 4$$

```
x = 8
print(x << 1)  # 16
print(x >> 1)  # 4
```

Operator Precedence

When several operators appear in one expression, Python follows an order of evaluation.
Example:

```
result = 10 + 5 * 2
```

Multiplication is evaluated before addition:

$$10 + (5 \times 2) = 20$$

It is not:

$$(10 + 5) \times 2 = 30$$

Best practice

Use parentheses when you want to make the intended order obvious.

Basic Operator Precedence

A simplified order is:

1. Parentheses
2. Exponentiation
3. Multiplication, division, floor division, modulus
4. Addition and subtraction
5. Comparisons
6. Logical operators

Example:

```
result = (10 + 2) * 3
```

First:

$$10 + 2 = 12$$

Then:

$$12 \times 3 = 36$$

Precedence: More Examples

```
a = 2 + 3 * 4
```

$$a = 2 + (3 \times 4) = 14$$

```
b = (2 + 3) * 4
```

$$b = (2 + 3) \times 4 = 20$$

Another example:

```
c = 2 ** 3 * 2
```

Exponentiation is evaluated before multiplication:

$$c = (2^3) \times 2 = 16$$

Key Points to Remember

- A variable is a name referring to a value.
- = assigns a value; == compares values.
- Variable names can contain letters, digits and underscores.
- A variable name cannot start with a digit.
- Variable names are case-sensitive.
- Keywords cannot be used as ordinary variable names.
- Meaningful names improve readability.
- Python has several built-in data types.
- Type conversion changes a value from one type to another.
- Operators perform arithmetic, comparison, logical and other operations.

Python Fundamentals – Session 1 Summary

- Introduction to Python
- Variables
- Assignment and reassignment
- Variable naming rules
- Naming best practices
- Multiple assignment
- Data types
- Constants
- Type conversion
- User input
- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Membership and identity
- Bitwise operators
- Operator precedence